Contribution title

First Author[1][0000-1111-2222-3333], Second Author[2,3][1111-2222-3333-4444], and Third Author[3][2222–3333-4444-5555]

[1] Princeton University, Princeton NJ 08544, USA
[2] Springer Heidelberg, Tiergartenstr. 17, 69121 Heidelberg, Germany
lncs@springer.com
http://www.springer.com/gp/computer-science/lncs
[3] ABC Institute, Rupert-Karls-University Heidelberg, Heidelberg, Germany
{abc,lncs}@uni-heidelberg.de

Resumen The abstract should briefly summarize the contents of the paper in 150–250 words. This is the English version of the abstract that provides a comprehensive overview of the research conducted, the methodology used, and the main findings obtained.

Keywords: first keyword· second keyword· another keyword

Título de la contribución

Resumen En este artículo se aborda el problema de la explicabilidad en modelos de aprendizaje automático mediante métodos de feature attribution.
En particular, se estudia una variante de los Shapley values conocida como Asymmetric Shapley Values (ASV), que permite incorporar conocimiento causal en la explicación de modelos de forma model-agnostic. A partir del análisis de su complejidad, se demuestra que el cálculo exacto de ASV es polinomial en modelos cuya distribución de entrada está representada por una red bayesiana del tipo Naive Bayes, en contraste con SHAP, que es #P-hard aún en este caso restringido.
Con el objetivo de extender estos resultados a clases más generales de redes bayesianas, se introduce una noción de clases de equivalencia sobre los órdenes topológicos del grafo causal subyacente, lo cual permite reducir drásticamente el número de permutaciones necesarias para computar ASV. Se presenta un algoritmo polinomial en el número de clases para identificarlas, y se implementa un esquema de cómputo exacto de ASV basado en estas clases. Además, se propone un nuevo método para computar en tiempo polinomial la predicción esperada de un árbol de decisión, sobre una distribución dada por una red bayesiana arbitraria, permitiendo así evaluar el algoritmo desarrollado para el cómputo de ASV en estos modelos.

Por último, se propone un algoritmo aproximado para calcular el ASV en familias de DAG's causales del tipo polytree. Para ello, se desarrolla un algoritmo de muestreo aleatorio de órdenes topológicos de polytrees. Estos resultados respaldan la viabilidad del enfoque propuesto en estructuras causales realistas, y se contrastan empíricamente con SHAP tanto en precisión como en eficiencia computacional.

Keywords: explicabilidad, árboles de decisión, Asymmetric Shapley Values (ASV), Shap, polytrees, ordenes topológicos

1. Introducción

En los últimos años, las capacidades de los modelos de inteligencia artificial han crecido de forma acelerada, al igual que la complejidad de sus arquitecturas. Este avance permitió resolver tareas que antes parecían inalcanzables, como la resolución de problemas matemáticos complejos por parte de modelos de lenguaje et. al, 2024. Al mismo tiempo, los modelos actuales han alcanzado escalas muy superiores a las de hace algunos años, tanto en cantidad de parámetros como en complejidad estructural Epoch AI, 2024. Esta falta de transparencia es especialmente problemática en contextos sensibles, como diagnósticos médicos o decisiones de alto impacto, donde no alcanza con obtener una predicción correcta: también es fundamental comprender cómo fue alcanzada. En este contexto surge el área de *Explainable Artificial Intelligence* (XAI), cuyo objetivo es desarrollar métodos que permitan explicar las decisiones de los modelos.

Dentro del área de explicabilidad, uno de los enfoques más influyentes es el de *feature attribution*, donde se asigna a cada variable de entrada un puntaje que refleja cuánto contribuyó a una predicción. En esta línea, *SHAP* es uno de los métodos más conocidos Lundberg y Lee, 2017. Está basado en los *Shapley values*, un concepto originado en teoría de juegos cooperativos Shapley et al., 1953. Intuitivamente, los Shapley values permiten repartir de manera justa el valor total obtenido por una coalición entre sus participantes, según cuánto aporta cada uno en promedio al incorporarse. Trasladado al aprendizaje automático, las features juegan el rol de los jugadores, y el valor asignado a cada una busca cuantificar cuánto contribuye a la predicción del modelo para una instancia particular.

A pesar de su popularidad, SHAP presenta limitaciones importantes. Por un lado, su cálculo exacto suele ser costoso, ya que requiere considerar una cantidad exponencial de subconjuntos o permutaciones de features. De hecho, existen resultados que muestran que calcular SHAP resulta intratable incluso para modelos o distribuciones relativamente simples Arenas et al., 2021, 2023; Marzouk et al., 2025; Van den Broeck et al., 2022. Por otro lado, también existen cuestionamientos sobre la calidad de las explicaciones que produce. En particular,

no siempre resulta claro cómo interpretar en inteligencia artificial los axiomas heredados de la teoría de juegos, y varios trabajos señalan que estas explicaciones pueden fallar en reflejar adecuadamente relaciones de dependencia, relevancia o causalidad entre las variables Fryer et al., 2021; Huang y Marques-Silva, 2023; Marques-Silva, 2023. Más en general, tampoco existe un consenso absoluto sobre qué debe considerarse una "explicación" en IA: muchas de las técnicas actuales son aproximaciones pragmáticas a un problema mucho más amplio, relacionado con comprensión, causalidad e interpretación humana Lipton, 2017; Miller, 2019.

Una propuesta para superar algunas de estas limitaciones son los *Asymmetric Shapley Values* (ASV), introducidos por Frye et al. Frye et al., 2019. Esta variante incorpora información causal al cálculo de los puntajes, restringiendo la atención a aquellas permutaciones de variables que respetan un grafo causal. De este modo, en lugar de tratar a todas las features simétricamente, se priorizan aquellas que aparecen causalmente antes que otras. Esto puede producir explicaciones más fieles al proceso generador de los datos y ayudar a detectar mejor fenómenos como sesgos o discriminación. Además, aunque ASV fue propuesto principalmente para mejorar la calidad explicativa, surge naturalmente la pregunta de si esta restricción adicional también puede traer beneficios desde el punto de vista computacional.

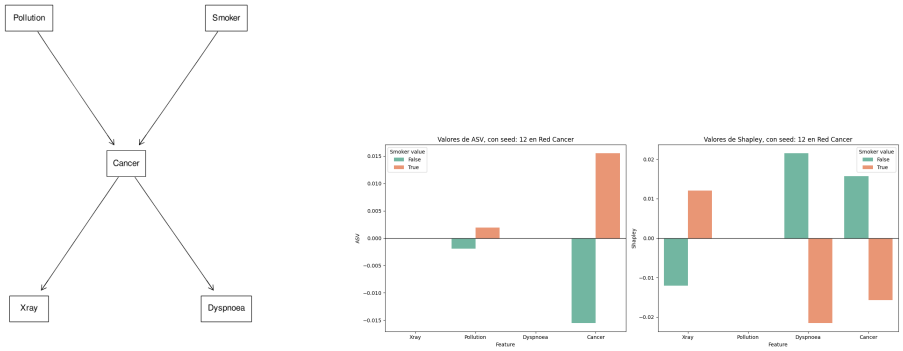
En esta tesis estudiamos el problema de calcular ASV cuando la distribución de los datos está representada por una red bayesiana. En particular, analizamos tanto el cómputo exacto como aproximado de ASV, considerando a la red bayesiana también como grafo causal subyacente. Mostramos que, en el caso de distribuciones Naive Bayes, ASV puede calcularse en tiempo polinomial en escenarios donde SHAP resulta intratable Van den Broeck et al., 2022. A partir de este resultado, nos enfocamos en árboles de decisión y en el desarrollo de algoritmos para calcular eficientemente predicciones promedio y explicaciones causales en estructuras más generales.

Echu: Esto se puede cambiar todavía, la verdad no tenemos muy clara cómo va a ser la estructura así que lo dejo cómo placeholder.

El resto del trabajo se organiza de la siguiente manera. En la Sección 2 introducimos las definiciones formales necesarias sobre SHAP, ASV, clasificadores y distribuciones probabilísticas. En la Sección 3 presentamos los principales resultados teóricos y de complejidad, junto con los algoritmos desarrollados. Luego, en la Sección 4 mostramos resultados experimentales que ilustran y validan el enfoque propuesto. Finalmente, en la Sección 5 discutimos las conclusiones y posibles líneas de trabajo futuro.

1.1. Explicaciones basadas en causalidad: un ejemplo con la red Cancer

Para ilustrar cómo la información causal puede afectar las explicaciones de un modelo, consideramos la red bayesiana *Cancer*, mostrada en la Figura 1a. En esta red, las variables `Pollution` y `Smoker` influyen sobre `Cancer`, mientras que `Xray` y `Dyspnoea` son consecuencias de esta última.



(a) Red bayesiana *Cancer*.

(b) Comparación entre SHAP y ASV.

Figura 1: Ejemplo en la red *Cancer*. SHAP puede asignar relevancia a variables que son consecuencias de otras (como `Xray` o `Dyspnoea`), mientras que ASV prioriza variables causalmente más cercanas al fenómeno.

A partir de datos sampleados de esta red entrenamos un árbol de decisión para predecir la variable `Smoker`. En este contexto podemos comparar dos formas de explicar una predicción: los valores de SHAP, que consideran todas las permutaciones posibles de features, y los Asymmetric Shapley Values (ASV), que restringen el cálculo a órdenes topológicos compatibles con la estructura causal del grafo.

La Figura 1b muestra un ejemplo de esta comparación. Se observa que SHAP asigna relevancia considerable a variables como `Xray` y `Dyspnoea`, que en la red son consecuencias de `Cancer`. Esto ocurre porque dichas variables pueden estar fuertemente correlacionadas con la predicción, aun cuando no sean causas directas del fenómeno. En cambio, ASV tiende a priorizar variables más cercanas a las causas del proceso generador de los datos, reduciendo la importancia de variables descendientes.

Este ejemplo ilustra cómo incorporar información causal puede producir explicaciones más alineadas con la estructura del problema. En redes donde se conoce (o se asume) una relación causal entre las variables, los ASVs permiten distinguir mejor entre variables que originan información relevante para la predicción y variables que simplemente reflejan efectos de otras.

1.2. Código fuente

Todos los algoritmos presentados en este trabajo fueron implementados. El código fuente puede ser encontrado online en el siguiente repositorio:

Repository Repository
Source code https://github.com/EchuCompa/pasantia-BICC

2. Definiciones

En esta sección introducimos las nociones formales que utilizaremos a lo largo del trabajo. Primero definimos los *Shapley values* y su instanciación al problema de *feature attribution*. Luego presentamos los *Asymmetric Shapley Values* (ASV), que incorporan relaciones entre las variables a la atribución mediante un grafo causal. Finalmente, introducimos las redes bayesianas como modelo para la distribución de los datos y describimos las familias restringidas de redes que consideraremos en nuestros resultados.

2.1. Shapley values y feature attribution

Trabajaremos con clasificadores binarios y features binarias. Esta restricción simplifica la notación sin afectar las ideas centrales ni los algoritmos desarrollados, que pueden extenderse naturalmente a dominios finitos más generales.

Sea X un conjunto finito de features. Una *entidad* sobre X es una función $e : X \to \{0, 1\}$, donde $e(x)$ representa el valor que la entidad e toma en el feature x. Denotamos por $\texttt{ent}(X)$ al conjunto de todas las entidades posibles sobre X, y asumimos dada una distribución de probabilidad Pr sobre $\texttt{ent}(X)$. Un *clasificador binario* es una función $M : \texttt{ent}(X) \to \{0, 1\}$.

Dado un modelo M y una entidad e, un *feature attribution score* es una función $\phi : X \to \mathbb{R}$, donde $\phi(x)$ mide la relevancia del feature x respecto de la predicción $M(e)$.

Uno de los métodos de atribución más estudiados es el SHAP-SCORE Lundberg y Lee, 2017, basado en los *Shapley values* Shapley et al., 1953. En teoría de juegos cooperativos, los Shapley values describen cómo repartir el valor total obtenido por una coalición entre sus jugadores. Más precisamente, dado un conjunto finito de jugadores $\mathcal{I}$, una *función característica* es una función

$$\nu : \mathcal{P}(\mathcal{I}) \to \mathbb{R}$$

que asigna un valor a cada coalición. Los Shapley values son la única asignación que satisface simultáneamente un conjunto de axiomas clásicos: *eficiencia*, *simetría*, *jugador nulo* y *linealidad*. Una forma cerrada de definirlos es la siguiente.

Definition 1 (Shapley values). *Sea $\mathcal{I}$ un conjunto finito y sea $\nu : \mathcal{P}(\mathcal{I}) \to \mathbb{R}$ una función característica. Para cada jugador $i \in \mathcal{I}$, su valor de Shapley se define como*

$$\phi_i(\nu) = \frac{1}{|\mathcal{I}|!} \sum_{\pi \in perm(\mathcal{I})} \left[\nu(\pi_{<i} \cup \{i\}) - \nu(\pi_{<i})\right], \tag{1}$$

donde $perm(\mathcal{I})$ es el conjunto de todas las permutaciones de $\mathcal{I}$, y $\pi_{<i}$ denota el conjunto de jugadores que aparecen antes que i en la permutación π.

Intuitivamente, esta expresión promedia la contribución marginal de un jugador sobre todos los órdenes posibles de llegada al juego. De manera equivalente, puede escribirse como

$$\phi_i(\nu) = \sum_{S \subseteq \mathcal{I} \setminus \{i\}} c_{|S|} \left[\nu(S \cup \{i\}) - \nu(S) \right],$$

donde

$$c_m = \frac{m!(|\mathcal{I}| - m - 1)!}{|\mathcal{I}|!}.$$

En aprendizaje automático, el conjunto de jugadores se identifica con el conjunto de features X, en nuestro ejemplo de la red *Cancer*, X sería el conjunto `Cancer, Pollution, Dyspnoea, Xray`. Dada una entidad e y un subconjunto $S \subseteq X$, queremos evaluar el comportamiento promedio del modelo al fijar los valores de las variables de S según la entidad e y marginalizar el resto de acuerdo con la distribución de los datos.

Para ello, definimos primero el conjunto de entidades *consistentes* con e en S:

$$\mathtt{cw}(e, S) = \{e' \in \mathtt{ent}(X) : e'(x) = e(x) \text{ para todo } x \in S\}.$$

La probabilidad condicionada a este conjunto queda dada por

$$\Pr[e' \mid \mathtt{cw}(e, S)] = \begin{cases} \dfrac{\Pr[e']}{\sum\limits_{e'' \in \mathtt{cw}(e,S)} \Pr[e'']} & \text{si } e' \in \mathtt{cw}(e, S), \\ 0 & \text{en caso contrario.} \end{cases}$$

Con esta notación, la función característica asociada a M, e y Pr se define como la predicción promedio condicionada a los valores fijados en S.

Definition 2 (Función característica inducida por un modelo). *Dados un modelo M, una entidad e y una distribución* Pr *sobre* $\mathtt{ent}(X)$*, definimos*

$$\nu_{M,e,\Pr}(S) = \sum_{e' \in \mathtt{cw}(e,S)} \Pr[e' \mid \mathtt{cw}(e, S)]\, M(e'). \tag{2}$$

Esto nos permite instanciar directamente los Shapley values en el problema de *feature attribution*.

Definition 3 (SHAP). *Dado un modelo M, una entidad e, una distribución* Pr *y un feature $x_i \in X$, el valor SHAP de x_i se define como*

$$Shap_{M,e,\Pr}(x_i) = \sum_{S \subseteq X \setminus \{x_i\}} c_{|S|} \left[\nu_{M,e,\Pr}(S \cup \{x_i\}) - \nu_{M,e,\Pr}(S) \right].$$

Nótese que los axiomas que estos valores satisfacen no tienen un significado claro en el contexto de la inteligencia artificial, ya que dependen de la definición de $\nu_{M,e,\Pr}$ Fryer et al., 2021. Además, para algunas nociones simples y robustas de *feature attribution* basadas en *explicaciones abductivas* , los Shapley values no logran asignar un puntaje de 0 a features irrelevantes Huang y Marques-Silva, 2023; Marques-Silva, 2023. Aun así, constituyen uno de los marcos más influyentes para estudiar explicaciones locales basadas en atribución.

2.2. Asymmetric Shapley Values y grafos causales

Echu: No hablamos mucho de la complejidad de Shap o de ASV, lo queremos hacer acá y contar los resultados previos o la idea es no contarlo y decirlo de una en los resultados teóricos?

La definición anterior trata a todas las permutaciones de features de manera uniforme. Sin embargo, si se dispone de un grafo causal sobre las variables, resulta natural restringir la atención a aquellas permutaciones compatibles con dicha estructura.

La generalización más simple consiste en reemplazar el promedio uniforme por una función de pesos arbitraria sobre las permutaciones.

Definition 4 (Shapley values ponderados). *Sea $w : perm(\mathcal{I}) \to \mathbb{R}$ una función tal que $\sum_{\pi \in perm(\mathcal{I})} w(\pi) = 1$. Definimos*

$$\phi_i^{assym}(\nu) = \sum_{\pi \in perm(\mathcal{I})} w(\pi) \left[\nu(\pi_{<i} \cup \{i\}) - \nu(\pi_{<i})\right]. \tag{3}$$

La familia de *Asymmetric Shapley Values* propuesta en Frye et al., 2019 surge al elegir la función de pesos a partir de un DAG que codifica relaciones causales entre las features.

Definition 5 (Asymmetric Shapley Values). *Sea $G = (X, E)$ un DAG cuyos nodos son los features. Denotemos por $topos(G)$ al conjunto de órdenes topológicos de G. Definimos la función de pesos*

$$w(\pi) = \begin{cases} \frac{1}{|topos(G)|} & \text{si } \pi \in topos(G), \\ 0 & \text{en otro caso.} \end{cases}$$

Entonces, dado un modelo M, una entidad e y una distribución $\Pr$, el valor ASV de un feature $x_i \in X$ se define como

$$Shap^{assym}_{M,e,\Pr}(x_i) = \frac{1}{|topos(G)|} \sum_{\pi \in topos(G)} \left[\nu_{M,e,\Pr}(\pi_{<i} \cup \{x_i\}) - \nu_{M,e,\Pr}(\pi_{<i})\right].$$

La intuición detrás de esta definición es que sólo se consideran órdenes que respetan la causalidad. De este modo, si una variable x es causa de una variable y, entonces x siempre aparece antes que y en las permutaciones admitidas. Esto tiende a favorecer a las variables causalmente anteriores y a reducir la importancia de variables que son meramente consecuencias de otras.

2.3. Redes bayesianas como modelo de distribución

Para evaluar los ASV no alcanza con fijar un modelo predictivo: también es necesario especificar una distribución sobre el espacio de entidades. En este trabajo utilizamos redes bayesianas para representar dicha distribución.

Definition 6 (Red bayesiana). *Una Red Bayesiana N sobre un conjunto de variables X es una tupla $N = (X, E, \Pr)$, donde (X, E) es un DAG y, para cada variable $x \in X$, se especifica una distribución condicional $\Pr(x \mid Parents(x))$. Estas distribuciones se representan mediante tablas de probabilidad condicional (*Conditional Probability Tables*, CPTs).*

La propiedad central de una red bayesiana es que factoriza la distribución conjunta usando únicamente dependencias locales. Si $X = \{X_1, \ldots, X_n\}$, entonces

$$\Pr(X_1, \ldots, X_n) = \prod_{i=1}^{n} \Pr(X_i \mid Parents(X_i)).$$

Esta factorización permite realizar inferencia probabilística de forma mucho más eficiente que manipulando directamente la distribución conjunta.

Conviene distinguir con cuidado entre *red causal* y *red bayesiana*. Una red bayesiana modela dependencias condicionales y define una distribución conjunta; un grafo causal, en cambio, pretende reflejar relaciones de causa y efecto entre variables. En general, ambas nociones no coinciden necesariamente. Sin embargo, en este trabajo asumiremos que el DAG subyacente a la red bayesiana puede utilizarse también como grafo causal para caclular los ASV. Esta decisión responde a dos motivos: por un lado, es consistente con el tipo de datos y experimentos que analizamos; por otro, nos permite estudiar simultáneamente el costo de la inferencia probabilística y el efecto de imponer restricciones causales en el cálculo de explicaciones.

Más precisamente, dado un modelo M, una entidad e y una red bayesiana N sobre las features, utilizaremos la distribución inducida por N para calcular la función característica $\nu_{M,e,\Pr}$, y el DAG de N como estructura causal para definir $Shap^{assym}_{M,e,\Pr}$.

2.4. Familias restringidas de redes bayesianas

La complejidad de la inferencia en redes bayesianas depende fuertemente de la estructura del grafo subyacente. En redes generales, calcular probabilidades marginales o condicionadas es un problema #P-HARD, mientras que para ciertas familias restringidas la inferencia es tratable. Por esta razón nos centramos en grafos para los cuáles la inferencia puede realizarse en tiempo polinomial.

La primera familia que consideramos es la de las redes *Naive Bayes*. En este caso, existe una variable raíz X_1 tal que todas las demás variables son sus hijas, y no hay otras aristas. Es decir, el conjunto de aristas es $E = \{(X_1, X_j) : 2 \leq j \leq n\}$.. La distribución conjunta factoriza entonces como

$$\Pr[e] = \Pr[X_1 = e(X_1)] \prod_{j=2}^{n} \Pr[X_j = e(X_j) \mid X_1 = e(X_1)].$$

Echu: Hace falta mostrar la fórmula de la probabilidad o mejor mostrar la imagen de que estructure tiene y listo? Igual la imagen ocupa espacio y no suma tanto, tal vez no hace falta ninguna de las dos.

Esta familia es especialmente relevante porque, aunque su estructura es muy simple, ya permite exhibir una diferencia fundamental entre SHAP y ASV: se sabe que calcular SHAP sobre distribuciones Naive Bayes es intratable incluso para modelos muy simples Van den Broeck et al., 2022, mientras que uno de nuestros resultados muestra que ASV puede ser tratable en este escenario.

La segunda familia central en este trabajo es la de los *polytrees*, es decir, DAGs cuyo grafo subyacente no dirigido es un árbol. Esta clase resulta importante porque admite inferencia probabilística en tiempo polinomial Pearl, 1986. Como el cálculo de la función característica involucra predicciones promedio condicionadas a evidencia, la posibilidad de realizar inferencia eficiente en la distribución de entrada es un ingrediente fundamental para obtener algoritmos tratables para ASV.

Echu: ¿Se dice tratable o tractable?

En síntesis, a lo largo de esta tesis trabajaremos principalmente con dos tipos de restricciones estructurales sobre la distribución de los datos: redes Naive Bayes, que sirven como caso base para nuestros resultados de tractabilidad, y polytrees, que constituyen la clase más general sobre la que desarrollamos algoritmos eficientes para inferencia y cómputo de explicaciones.

Echu: ¿Este parrafo juega o este de más?

3. Resultados teóricos

3.1. Tratabilidad de ASV

Como vimos en la sección anterior, la complejidad de la distribución de entrada no es el único factor que determina la dificultad de calcular explicaciones basadas en Shapley values. De hecho, aun para distribuciones estructuralmente simples como las redes Naive Bayes, el cálculo exacto de SHAP sigue siendo intratable Van den Broeck et al., 2022. En contraste, los Asymmetric Shapley Values (ASV) restringen las permutaciones consideradas a aquellas compatibles con el grafo causal. En esta sección mostramos que esta restricción permite obtener tractabilidad en casos donde SHAP no la tiene.

Consideramos el caso en que la distribución de los datos está dada por una red bayesiana Naive Bayes, y tomamos su DAG subyacente como grafo causal para calcular ASV. Recordemos que, en este caso, toda permutación topológica válida debe ubicar a la variable raíz X_1 antes que cualquier otra variable.

Esta observación permite caracterizar explícitamente el conjunto de órdenes topológicos y explotar la estructura de la distribución en el cálculo de ASV.

Theorem 1. *Sea $\mathcal{F}$ una familia de modelos. Los Asymmetric Shapley Values pueden calcularse en tiempo polinomial para distribuciones Naive Bayes si y solo si los Shapley values pueden calcularse en tiempo polinomial para $\mathcal{F}$ bajo distribuciones producto.*

Echu: La demo es larga, la ponemos completa o damos una intuición la misma? Habría que ver el tema de si podemos mandar cosas al apéndice.

Demostración. Primero, probamos la implicación de derecha a izquierda. Sea x_1 el padre de todos los demás nodos en el DAG. Vamos a mostrar cómo calcular $Shap^{assym}_{M,e,\Pr}(x_j)$ para cualquier $2 \leq j \leq n$ y $Shap^{assym}_{M,e,\Pr}(x_1)$ de forma independiente.

Observemos que el DAG tiene $(n-1)!$ órdenes topológicos, uno para cada permutación de los features $\{x_2, \ldots, x_n\}$, y $\pi(x_1) = 1$ para todas ellas. Entonces,

$$Shap^{assym}_{M,e,\Pr}(X_j) = \sum_{\pi \in topos(G)} w(\pi) \left[\nu_{M,e,\Pr}(\pi_{<j} \cup \{x_j\}) - \nu_{M,e,\Pr}(\pi_{<j})\right]$$

es equivalente a:

$$Shap^{assym}_{M,e,\Pr}(X_j) = \frac{1}{(n-1)!} \sum_{\pi \in perm(\{x_2,\ldots,x_n\})} \left[\nu_{M,e,\Pr}(\{x_1\} \cup \pi_{<j} \cup \{x_j\}) - \nu_{M,e,\Pr}(\{x_1\} \cup \pi_{<j})\right]$$

Así, una vez que x_1 está fijado, la distribución para las variables $x_2, \ldots, x_n$ es una distribución producto con $p_{x_j} = P(X_j = 1 | X_1 = e(x_1))$. Para simplificar, asumamos que $e(x_1) = 1$, y consideremos la distribución de producto $\Pr'$ definida como:

$$\Pr{}'[X_i = 1] = p_i = \begin{cases} 1 & i = 1 \\ \Pr[X_i = 1 | X_1 = e(x_1)] & \text{en otro caso} \end{cases}$$

que intuitivamente se obtiene de Pr fijando $X_1 = 1$. Siempre que x_1 esté fijo, ambas distribuciones Pr y $\Pr'$ se comportan de la misma forma:

Lemma 1. *Para cualquier $S \subseteq X \setminus \{x_1\}$, se cumple que:*

$$\nu_{M,e,\Pr}(\{x_1\} \cup S) = \nu_{M,e,\Pr'}(\{x_1\} \cup S) = \nu_{M,e,\Pr'}(S)$$

Demostración. Esto se sigue de directamente manipulando la expresión:

$$\begin{aligned}
\nu_{M,e,\Pr}(\{x_1\} \cup S) &= \sum_{e' \in \texttt{cw}(e,\{x_1\} \cup S)} \Pr[e'|S] M(e') \\
&= \sum_{e' \in \texttt{cw}(e,\{x_1\} \cup S)} \left(\prod_{\substack{e'(y)=1 \\ y \notin \{x_1\} \cup S}} p_y \prod_{\substack{e'(y)=0 \\ y \notin \{x_1\} \cup S}} (1 - p_y) \right) M(e') \quad (4) \\
&= \sum_{e' \in \texttt{cw}(e,S)} \left(\prod_{\substack{e'(y)=1 \\ y \notin S}} p_y \prod_{\substack{e'(y)=0 \\ y \notin S}} (1 - p_y) \right) M(e') \\
&= \sum_{e' \in \texttt{cw}(e,S)} \Pr{}'[e'|S] M(e') \\
&= \nu_{M,e,\Pr'}(S)
\end{aligned}$$

donde la segunda igualdad viene de reemplazar Pr por la distribución producto y la tercera igualdad se deduce al observar que para todas las entidades $e' \in \texttt{cw}(e, S)$ tales que $e'(x_1) = 0$ se cumple que

$$\prod_{\substack{e'(y)=1 \\ y \notin S}} p_y \prod_{\substack{e'(y)=0 \\ y \notin S}} (1 - p_y) = 0$$

Además, la segunda ecuación es igual a $\nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup S)$.

Usando este lema, ahora demostramos

$$Shap^{assym}_{M,e,\mathrm{Pr}}(x_j) = Shap_{M,e,\mathrm{Pr}'}(x_j) \tag{5}$$

Se cumple que

$$\begin{aligned}
Shap_{M,e,\mathrm{Pr}'}(x_j) &= \frac{1}{n!} \sum_{\pi \in perm(X)} [\nu_{M,e,\mathrm{Pr}'}(\pi_{<j} \cup \{x_j\}) - \nu_{M,e,\mathrm{Pr}'}(\pi_{<j})] \\
&= \frac{1}{n!} \sum_{\pi \in perm(X)} [\nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup \pi_{<j} \cup \{x_j\}) - \nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup \pi_{<j})] \\
&= \frac{n}{n!} \sum_{\pi \in perm(\{x_2,\ldots,x_n\})} [\nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup \pi_{<j} \cup \{x_j\}) - \nu_{M,e,\mathrm{Pr}'}(\{x_1\} \cup \pi_{<j})] \\
&= \frac{1}{n-1!} \sum_{\pi \in perm(\{x_2,\ldots,x_n\})} [\nu_{M,e,\mathrm{Pr}}(\{x_1\} \cup \pi_{<j} \cup \{x_j\}) - \nu_{M,e,\mathrm{Pr}}(\{x_1\} \cup \pi_{<j})] \\
&= Shap^{assym}_{M,e,\mathrm{Pr}}(x_j)
\end{aligned}$$

donde la segunda y cuarta igualdad se siguen del Lema 1, la última igualdad de la Ecuación 3.1 y la tercera al observar que para cada permutación de $perm(\{x_2, \ldots, x_n\})$ podemos construir n permutaciones de $perm(X)$ insertando x_1 en todos los lugares posibles, y que para cada una de estas permutaciones la expresión dentro de la suma es la misma. Así, la Ecuación 5 muestra que calcular $Shap^{assym}_{M,e,\mathrm{Pr}}(x_j)$ se reduce a calcular los valores Shapley habituales para una distribución independiente particular.

Ahora consideramos $Shap^{assym}_{M,e,\mathrm{Pr}}(x_1)$. Observemos que

$$\begin{aligned}
Shap^{assym}_{M,e,\mathrm{Pr}}(x_1) &= \frac{1}{|topos(G)|} \sum_{\pi \in topos(G)} [\nu_{M,e,\mathrm{Pr}}(\pi_{<1} \cup \{x_1\}) - \nu_{M,e,\mathrm{Pr}}(\pi_{<1})] \\
&= \frac{1}{|topos(G)|} (\nu_{M,e,\mathrm{Pr}}(\{x_1\}) - \nu_{M,e,\mathrm{Pr}}(\emptyset))
\end{aligned}$$

ya que para todas las permutaciones el conjunto $\pi_{<1} = \emptyset$. Además, sabemos que $\nu_{M,e,\mathrm{Pr}}(\{x_1\}) = \nu_{M,e,\mathrm{Pr}'}(\emptyset)$ por el Lema 1, y que

$$
\begin{aligned}
\nu_{M,e,\Pr}(\emptyset) &= \sum_{e' \in \texttt{ent}(X)} \Pr[e] M(e) \\
&= \sum_{e' \in \texttt{ent}(X) \quad e'(x_1)=1} \Pr[e'|e'(x_1)=1] \Pr[X_1=1] M(e') + \sum_{\substack{e' \in \texttt{ent}(X) \\ e'(x_1)=0}} \Pr[e'|e'(x_1)=1] \Pr[X_1=0] M(e') \\
&= \Pr[X_1=1] \nu_{M,e,\Pr'}(\emptyset) + \Pr[X_1=0] \nu_{M,e,\Pr''}(\emptyset)
\end{aligned}
$$

donde la distribución $\Pr''$ es la distribución producto obtenida al fijar $X_1 = 0$ como

$$
\Pr{}''[X_i = 1] = \begin{cases} 0 & i = 0 \\ Pr[X_i = 1 | X_1 = 0] & \text{de otro modo} \end{cases}
$$

Finalmente,

$$
Shap^{assym}_{M,e,\Pr}(x_1) = \frac{1}{|topos(G)|}((1 - \Pr[X_1 = 1]) \nu_{M,e,\Pr'}(\emptyset) - \Pr[X_1 = 0] \nu_{M,e,\Pr''}(\emptyset))
$$

y reducimos el problema de calcular $Shap^{assym}_{M,e,\Pr}(x_1)$ al de calcular la predicción promedio del modelo M para dos distribuciones independientes diferentes $\Pr'$ y $\Pr''$. Según Van den Broeck et al., 2022[Teorema 1], se cumple que estos promedios pueden ser calculados en tiempo polinomial si y sólo si los valores de Shapley pueden ser calculados en tiempo polinomial para una distribución de producto arbitraria.

La demostración de izquierda a derecha sigue al observar que una distribución producto es un caso particular de una Distribución Naive Bayes en la cual, para cualquier valor del padre X_1, las distribuciones condicionales son las mismas.

El Teorema 1 muestra que, a diferencia de SHAP, ASV puede ser tratable al explotar la estructura causal. En particular, esta tratabilidad depende de poder calcular eficientemente la predicción promedio. En la experimentación nos enfocamos en árboles de decisión, donde desarrollamos algoritmos polinomiales para esta tarea.

Echu: Tiene sentido el teaser de la experimentación? O saco esa oración?

3.2. Clases de equivalencia y relación con los órdenes topológicos

Recordemos la definición de ASV:

$$
Shap^{assym}_{M,e,\Pr}(x_i) = \frac{1}{|topos(G)|} \sum_{\pi \in topos(G)} \left[\nu_{M,e,\Pr}(\pi_{<i} \cup \{x_i\}) - \nu_{M,e,\Pr}(\pi_{<i}) \right].
$$

Para simplificar la notación, en esta sección tomamos M, e y $\Pr$ fijos, y escribimos simplemente ν en lugar de $\nu_{M,e,\Pr}$. La idea es reducir la cantidad

de órdenes topológicos sobre los cuales evaluamos ν, agrupando aquellos que devuelven el mismo resultado al evaluarse sobre ν.

La relación ideal sería identificar dos órdenes topológicos $\pi^1, \pi^2 \in topos(G)$ cuando

$$\nu(\pi^1_{<i}) = \nu(\pi^2_{<i}),$$

pero esta condición es costosa de verificar, ya que requiere evaluar explícitamente la función característica. Por este motivo trabajamos con una relación más fina, puramente combinatoria.

Definition 7. *Sea $G = (V, E)$ un DAG y sea $x_i \in V$. Definimos la relación R sobre $topos(G)$ por*

$$\pi^1 R \pi^2 \iff \{\pi^1_{<i}\} = \{\pi^2_{<i}\}.$$

Es decir, dos órdenes topológicos son equivalentes si tienen exactamente el mismo conjunto de nodos a la izquierda de x_i.

Esta relación depende únicamente de la posición relativa de los nodos respecto de x_i. Para representarla, resulta útil dividir los nodos del grafo en ancestros, descendientes y nodos no relacionados con x_i.

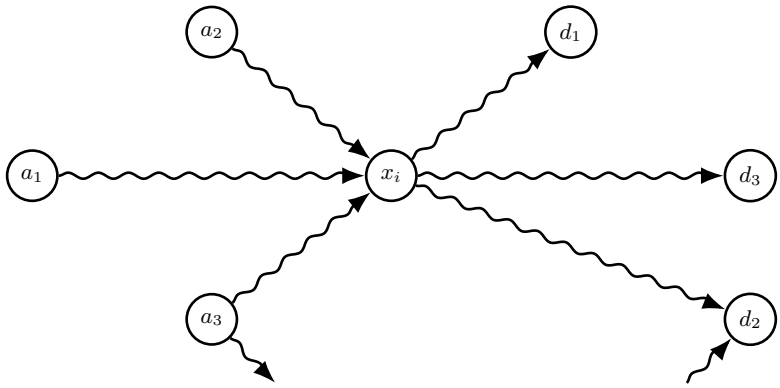


Figura 2: Al fijar un nodo x_i, dividimos el resto en tres grupos: ancestros (a), descendientes (d) y nodos no relacionados. Los ancestros siempre quedan a la izquierda de x_i, los descendientes siempre a la derecha, y solo los nodos no relacionados no tienen una posición fija respecto de x_i entre distintos órdenes topológicos.

Definition 8. *Sea $G = (V, E)$ un DAG y $x_i \in V$. Una clase de equivalencia $[\pi]_R$ queda determinada por una función*

$$f_\pi : V \setminus \{x_i\} \to \{left, right\},$$

donde $f_\pi(v)$ indica si el nodo v aparece a la izquierda o a la derecha de x_i en un orden topológico π. Denotamos por $L([\pi]_R)$ y $R([\pi]_R)$ a los conjuntos de nodos etiquetados con left y right, respectivamente.

Proposition 1. *Sea $eqCl(G, x_i)$ el conjunto de clases de equivalencia inducidas por R. Entonces*

$$Shap^{assym}_{M,e,\Pr}(x_i) = \frac{1}{|topos(G)|} \sum_{[\pi]_R \in eqCl(G,x_i)} |[\pi]_R| \left[\nu(L([\pi]_R) \cup \{x_i\}) - \nu(L([\pi]_R))\right].$$

Demostración. Basta particionar la suma original de ASV por clases de equivalencia. Dentro de una misma clase, todos los órdenes topológicos inducen el mismo conjunto de nodos antes de x_i, y por lo tanto la expresión entre corchetes es constante.

Echu: ¿Es muy vende humo ponerle demostración? Sino lo escribo cómo un parrafo y le pongo que es la intuición.

La ventaja de estas clases es que no requiere evaluar ν para decidir si dos órdenes pertenecen a la misma. La Proposición 1 muestra que, si podemos calcular eficientemente las clases de equivalencia y sus tamaños, entonces podemos computar ASV sin enumerar todos los órdenes topológicos.

3.3. Algoritmo exacto para el cálculo de clases de equivalencia

Nuestro objetivo es computar $Shap^{assym}_{M,e,\Pr}(x_i)$ dado un grafo causal G, el cual puede expresarse como una suma sobre clases de equivalencia de órdenes topológicos respecto de x_i. En consecuencia, el problema se reduce a calcular el conjunto $eqClassSizes(G, x_i)$, es decir, las clases de equivalencia junto con sus tamaños.

Enfoque naive Un enfoque directo consiste en generar todos los órdenes topológicos de G Knuth y Szwarcfiter, 1974 y luego agruparlos en clases de equivalencia según qué nodos no relacionados aparecen antes de x_i. A partir de esta agrupación, se obtiene un representante por clase y su tamaño.

Si bien este procedimiento es correcto, resulta computacionalmente ineficiente. En el peor caso, el número de órdenes topológicos crece factorialmente en el tamaño del grafo, es decir, $O(n!)$, lo cual vuelve impracticable este enfoque incluso para instancias moderadas. Esto motiva la necesidad de un algoritmo exacto que evite enumerar explícitamente todos los órdenes.

Estructura del *dtree* y estrategia general Proponemos un algoritmo recursivo que explota la estructura del *dtree* inducido por el DAG alrededor del nodo x_i. Particionamos los nodos en tres conjuntos: ancestros A, descendientes D y nodos no relacionados. Dentro de estos últimos, consideramos los subárboles cuyas raíces pertenecen al conjunto UR (unrelated roots).

La idea central es descomponer el problema en dos etapas:

1. calcular las clases de equivalencia de cada árbol no relacionado de manera independiente;

2. combinar dichas clases con los ancestros y descendientes para obtener las clases completas.

Cada clase de equivalencia se representa mediante una tupla

$$(\mathsf{Rep}([\pi]_R), leftTopos, rightTopos),$$

donde $leftTopos$ y $rightTopos$ representan la cantidad de órdenes topológicos posibles a la izquierda y derecha de x_i, respectivamente. El tamaño de la clase es entonces $leftTopos \cdot rightTopos$.

La Figura 3 ilustra esta partición.

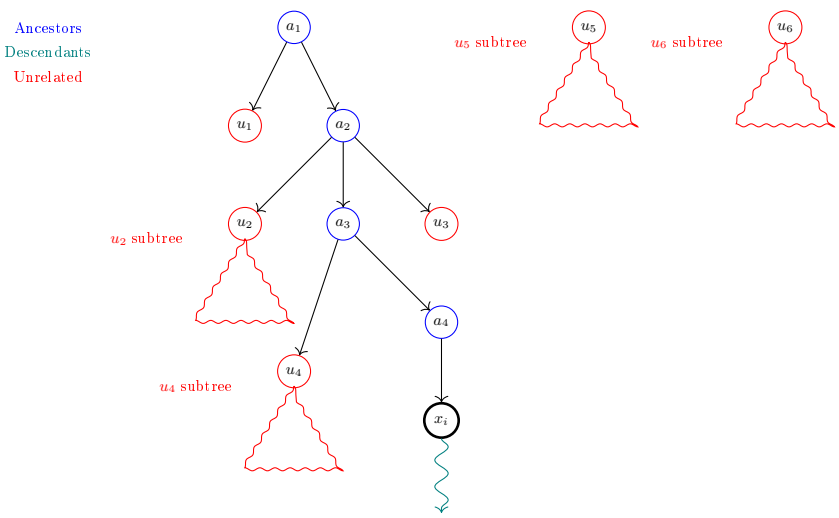


Figura 3: Partición del grafo en ancestros, descendientes y árboles no relacionados respecto de x_i.

Clases de equivalencia de árboles no relacionados Sea UR el conjunto de raíces de árboles no relacionados. Para cada $ur \in UR$, calculamos recursivamente sus clases de equivalencia mediante la función $UnrEC$.

El caso base ocurre cuando n es una hoja:

$$UnrEC(n) = \{(\{n_l\}, 1, 1), (\{n_r\}, 1, 1)\}.$$

Para un nodo interno, combinamos las clases de equivalencia de sus hijos y consideramos dos posibilidades: ubicar n a la izquierda o a la derecha de x_i. Esto da lugar a la siguiente recurrencia:

$$UnrEC(n) = \begin{cases} \{(\{n_l\}, 1, 1), (\{n_r\}, 1, 1)\} & \text{si } n \text{ es hoja} \\ \bigcup_{\forall mix} union(mix, n_{left}) \ \cup \ union(right, n_{right}) & \text{en otro caso} \end{cases} \tag{6}$$

Existe una correspondencia biyectiva entre las clases de equivalencia de un nodo y las combinaciones de clases de sus hijos, lo cual garantiza la correctitud del esquema recursivo.

Combinación con descendientes Los descendientes de x_i siempre aparecen a la derecha, por lo que pueden tratarse como un bloque independiente. El número de órdenes topológicos de un *dtree* enraizado en n está dado por:

$$topoSorts(n) = \binom{\sum_j |T_{n_j}|}{|T_{n_1}|, \ldots, |T_{n_{|n|}}|} \prod_j topoSorts(n_j), \tag{7}$$

lo cual refleja la intercalación de los órdenes de los subárboles hijos.

Combinación con ancestros

Echu: ¿Debería explicar un poco menos en esta sección y meter menos fórmulas? La puedo escribir más por arriba.

A diferencia de los descendientes, los ancestros imponen restricciones de precedencia. No es posible intercalar libremente nodos no relacionados con ancestros mediante coeficientes multinomiales.

Para resolver este problema definimos la función $leftOrders$, que contabiliza cuántas formas válidas existen de intercalar los nodos no relacionados (ubicados a la izquierda) con los ancestros, respetando las dependencias del DAG.

La Figura 4 ilustra una posible disposición de los ancestros.

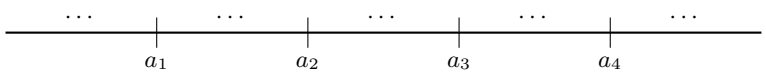


Figura 4: Orden relativo de los ancestros.

Algorithm 1 leftOrders

1. Determinar la posición del siguiente ancestro.
2. Elegir nodos disponibles para ubicar antes de dicha posición.
3. Actualizar el conjunto de nodos restantes.
4. Continuar recursivamente.

Fórmula final Integrando ambos componentes, obtenemos que el conjunto completo de clases de equivalencia puede construirse a partir del producto cartesiano de las clases de equivalencia de los árboles no relacionados. Formalmente,

$$eqClassSizes(G, x_i) = \bigcup_{\forall mix \in UnrEC(ur_1) \times \cdots \times UnrEC(ur_{|UR|})} (eqCl(A, D, mix), eqClassTopos(A, D, mix)) \,. \tag{8}$$

Aquí, cada mix representa una elección de clases de equivalencia para cada árbol no relacionado. A partir de dicha elección, la función $eqCl(A, D, mix)$ construye la representación completa de la clase, mientras que $eqClassTopos(A, D, mix)$ calcula su tamaño combinando:

- la cantidad de intercalaciones válidas con los ancestros, dada por $leftOrders$;
- la cantidad de órdenes topológicos de los descendientes;
- y las contribuciones internas de los árboles no relacionados.

De este modo, la expresión anterior no solo enumera todas las clases de equivalencia, sino que también computa exactamente su cardinalidad, sin necesidad de generar explícitamente los órdenes topológicos.

Complejidad La complejidad total del algoritmo está dominada por dos componentes principales:

- el cálculo de $UnrEC$, que recorre recursivamente los árboles no relacionados y combina las clases de sus hijos;
- la combinación global de clases, que requiere procesar el producto cartesiano de dichas clases y, para cada combinación, computar $leftOrders$ y las contribuciones de los descendientes.

En consecuencia, el costo total puede expresarse como proporcional al número de combinaciones de clases de equivalencia generadas, multiplicado por el costo de combinarlas con ancestros y descendientes. En particular, si denotamos por k el número total de clases de equivalencia resultantes, el algoritmo tiene complejidad del orden de $O(k^3 \cdot \text{poly}(n))$.

Si bien k puede ser exponencial en el peor caso, este enfoque evita depender del número total de órdenes topológicos del grafo, que puede ser factorial. Por lo tanto, el algoritmo representa una mejora sustancial respecto del enfoque naive, especialmente en grafos donde el número de clases de equivalencia es significativamente menor que el número de órdenes topológicos.

3.4. Aproximación de ASV mediante sampleo

En las secciones anteriores nos concentramos en algoritmos exactos para calcular ASV, ya sea enumerando todas las clases de equivalencia o explotando su estructura para reducir la cantidad de evaluaciones de la función característica. Sin embargo, cuando el número de clases o de órdenes topológicos es demasiado

grande, incluso estos enfoques pueden volverse prohibitivamente costosos. Esto motiva la consideración de algoritmos aproximados basados en técnicas de muestreo.

Siguiendo la idea de las clases de equivalencia, proponemos reemplazar la suma exacta que define ASV por un promedio empírico sobre una muestra aleatoria. Este enfoque permite estimar el valor de ASV con buena precisión utilizando una cantidad moderada de muestras, ya que el error del estimador decrece con el tamaño de la muestra.

Existen dos variantes naturales de esta estrategia. La primera consiste en samplear órdenes topológicos del DAG causal G y estimar el valor de ASV a partir de sus contribuciones marginales. La segunda consiste en samplear clases de equivalencia y utilizar un representante de cada clase para aproximar el promedio exacto.

En ambos casos, el punto central es disponer de un mecanismo que permita generar muestras con la distribución correcta (uniforme en el caso de órdenes topológicos, o proporcional al tamaño en el caso de clases de equivalencia). Bajo esta condición, el cálculo de ASV se reduce a promediar evaluaciones de la función característica sobre muestras i.i.d.

Una ventaja fundamental de este enfoque es que desacopla el cálculo de ASV de la estructura específica del grafo. En particular, mientras que los algoritmos exactos desarrollados anteriormente explotan propiedades estructurales como la de los *dtrees* o polytrees, el enfoque basado en sampleo puede aplicarse a *cualquier* DAG, siempre que se disponga de un procedimiento para generar muestras adecuadas. Esto amplía significativamente el alcance práctico del método.

El caso de órdenes topológicos resulta especialmente natural, ya que

$$Shap^{assym}_{M,e,\Pr}(x_i) = \frac{1}{|topos(G)|} \sum_{\pi \in topos(G)} [\nu_{M,e,\Pr}(\pi_{<i} \cup \{x_i\}) - \nu_{M,e,\Pr}(\pi_{<i})]$$

puede interpretarse como el valor esperado de la variable aleatoria

$$Z_\pi = \nu_{M,e,\Pr}(\pi_{<i} \cup \{x_i\}) - \nu_{M,e,\Pr}(\pi_{<i}), \qquad \pi \sim \mathrm{Unif}(topos(G)).$$

Por lo tanto, si podemos samplear órdenes topológicos uniformemente, obtenemos un estimador insesgado de ASV promediando copias i.i.d. de Z_π. Además al tener algoritmos de sampleo de órdenes topologicos para DAGs en general y no solo *dtrees*este algoritmo logra ser más general.

También a partir de estos órdenes sampleados podemos dividirlos por sus clases de equivalencia al igual que el algoritmo naive con probabilidad proporcional a su tamaño y estimar el valor mediante el promedio de sus representantes. Veamos a continuación dos posibles estrategias de sampleo.

Sampleo uniforme de órdenes topológicos Nos concentramos en el problema de samplear órdenes topológicos de un DAG de manera uniforme. Un primer enfoque consiste en generar todos los órdenes topológicos mediante un algoritmo determinístico, como el de Knuth Knuth y Szwarcfiter, 1974, y devolverlos a

medida que se producen. Sin embargo, este procedimiento no resulta adecuado para sampleo: al ser determinístico, no genera una distribución uniforme y puede sesgar la muestra cuando distintos nodos fuente aparecen con frecuencias muy diferentes como comienzo de un orden.

La idea correcta consiste en construir el orden de manera recursiva. Sea S el conjunto de nodos fuente de un DAG D. Para cada $s \in S$, definimos

$$p(s) = \frac{\#topos(D - \{s\})}{\#topos(D)},$$

donde $\#topos(D - \{s\})$ es la cantidad de órdenes topológicos que comienzan con s. El algoritmo selecciona el primer nodo del orden según esta distribución y luego continúa recursivamente sobre el grafo restante.

Este procedimiento garantiza que los órdenes topológicos se generan con la distribución uniforme deseada, siempre que se disponga de un método para contar la cantidad de órdenes de los subgrafos. En nuestro caso, esto se logra mediante el algoritmo de conteo desarrollado previamente para polytrees. La complejidad de este algoritmo es $O(k! \cdot n^k \cdot n^{3k-1}) = O(k! n^{4k-1})$, siendo k el grado máximo de un nodo.

Echu: ¿Agrego la imagen para ilustrar este algoritmo o al pedo? Me parece que no porque no le deberíamos dar mucha bola, no es tan relevante.

Aún así, el problema que tiene este algoritmo es que solo funciona para el caso de polytrees, y sabemos que además el conteo en DAGs generales es #P-HARD, por lo que buscamos otra estrategía de sampleo distinta.

Aún así, para el caso de DAGs generales utilizamos un método de sampleo basado en cadenas de Markov, siguiendo la idea de Huber Huber, 2006. El algoritmo considera como estados a los distintos órdenes topológicos del grafo y parte de un orden válido cualquiera. Luego, en cada paso, elige uniformemente una posición i y propone intercambiar los elementos ubicados en las posiciones consecutivas i e $i + 1$. Este intercambio se acepta con probabilidad $1/2$ únicamente si los dos nodos involucrados son incomparables en el DAG, es decir, si no existe un camino dirigido entre ellos; en caso contrario, el orden se mantiene. Para verificar esta condición de manera eficiente, se precomputa el cierre transitivo del grafo, de modo que cada chequeo de comparabilidad cuesta tiempo constante. Como la cadena siempre permanece dentro del conjunto de órdenes topológicos válidos y sus transiciones son simétricas, luego de una cantidad suficientemente grande de pasos de mezcla su distribución se aproxima a la uniforme sobre todos los órdenes topológicos. Su complejidad es de $O(n^3 \log_n)$, y en la práctica, esto permite obtener muestras aproximadamente uniformes sin necesidad de resolver el problema de conteo exacto, que en DAGs generales es $\sharp P$-hard.

Echu: Acá puedo contar lo que se vio en la experimentación. Primero, que para el caso de grado acotado nuestro algoritmo es más rápido y crece más lento el tiempo que tarda en el tamaño del grafo. Luego que al nuestro algoritmo reutilizar resultados también crece más lento el tiempo que tarda en base a la cantidad de sampleos, ya que el otro arranca de 0 cada vez. Pero que aún así el otro algoritmo es mejor porque para grafos de grado no acotado y grafos que no son polytrees funciona. O tal vez esto lo puedo mover para la parte de resultados experimentales.

Error del sampleo El esquema de aproximación basado en sampleo depende de dos subrutinas: la capacidad de generar muestras y la de evaluar la función

característica sobre cada una de ellas. En particular, el algoritmo consiste en samplear independientemente órdenes topológicos y promediar sus contribuciones marginales.

Más precisamente, si $\pi^{(1)}, \ldots, \pi^{(m)}$ son órdenes topológicos sampleados uniformemente, el estimador viene dado por

$$\widehat{ASV} = \frac{1}{m} \sum_{j=1}^{m} \left[\nu_{M,e,\mathrm{Pr}}(\pi^{(j)}_{<i} \cup \{x_i\}) - \nu_{M,e,\mathrm{Pr}}(\pi^{(j)}_{<i}) \right].$$

Cada muestra requiere generar un orden topológico y evaluar la función característica una cantidad constante de veces. La cantidad de muestras necesarias para obtener una buena aproximación depende de la precisión deseada y puede acotarse mediante resultados estándar de concentración, como se detalla en el apéndice.

Echu: ¿Hay apéndice? Si no hay apéndice, meto las demostraciones que tenemos sobre la estimación del error promedio acá?

De manera análoga, este enfoque puede aplicarse utilizando clases de equivalencia en lugar de órdenes topológicos, siempre que se puedan samplear con la distribución adecuada.

Echu: Queda medio raro meter esta sección acá, tal vez extraería la parte de la predicción promedio en una sección más y luego metería esto al final para poner algo de la experimentación.

3.5. Instanciaciones particulares

En nuestro caso, nos enfocamos en árboles de decisión como modelo M, y en distribuciones Pr representadas mediante redes bayesianas. En este contexto, desarrollamos un algoritmo para calcular

$$\nu_{M,e,\mathrm{Pr}}(S)$$

basado en recorrer el árbol de decisión y acumular las condiciones de camino inducidas por cada rama. Para cada hoja compatible con la evidencia, se computa su probabilidad mediante inferencia en la red bayesiana y se pondera por la predicción de la hoja. La predicción promedio se obtiene sumando estos aportes.

Cuando la red bayesiana tiene estructura de polytree, la inferencia puede realizarse en tiempo polinomial, por lo que el costo de evaluar la función característica es también polinomial.

Esto induce un esquema general para calcular ASV: siempre que el sampleo/generación exacta de órdenes topológicos o clases de equivalencia sea eficiente, se obtiene un algoritmo aproximado cuyo costo total es polinomial en el tamaño de la instancia y en los parámetros de precisión.

Por el contrario, cuando la predicción promedio no puede calcularse eficientemente, este enfoque deja de ser aplicable. En particular, el Teorema 1 sugiere que la tratabilidad de ASV está intrínsecamente ligada al costo de computar dichas esperanzas, ya que cada muestra requiere evaluar una contribución marginal.

En síntesis, los algoritmos propuestos resultan especialmente efectivos en escenarios donde:

1. el sampleo/conteo puede realizarse eficientemente;
2. la predicción promedio es computable en tiempo polinomial;
3. el número de órdenes o clases es demasiado grande para un enfoque exacto.

Este es precisamente el régimen considerado en la experimentación.

Echu: Acá no mostre nada de la experimentación que queríamos contar en teoría, siento que para el flow debería venir acá. ¿Que experimentos metemos? Yo banco "Clases de equivalencia vs Órdenes Topológicos" y "ASV exacto sin EqClasses vs ASV aproximado"

3.6. ASV exacto

Recordemos la fórmula introducida en la sección **??** para ASV, utilizando nuestra heurística.

$$\phi_i^{assym}(\nu) = \sum_{\pi \in perm(\mathcal{I})} w(\pi)\left[\nu(\pi_{<i} \cup i) - \nu(\pi_{<i})\right] =$$
$$\frac{1}{|topos(G)|} \sum_{[\pi]_R \in eqCl(G,x_i)} (\nu(\pi_{<i} \cup \{x_i\}) - \nu(\pi_{<i})) \cdot |[\pi]_R|$$

Dado un DAG G, un nodo $x_i \in V(G)$ y una función característica ν. Nuestro algoritmo completo queda así entonces:

- Calculamos $eqClass$ a través de $eqClassSizes(G, x_i)$, el algoritmo que introducimos en la sección 3.3
- Luego para cada clase de equivalencia vamos a calcular el promedio teniendo en cuenta los features previos a x_i. Para realizar el promedio vamos a utilizar el algoritmo de la sección 3.5. (En el caso de que el modelo no sea un árbol, podemos aproximarlo utilizando Monte Carlo u otro método)
- Por último sumamos los resultados para obtener él $Shap^{assym}$ para el feature i.

Las modificaciones que introducimos son para mejorar la performance del mismo, sin modificar los resultados obtenidos a diferencia del enfoque aproximado. En la sección a continuación vamos a ver cuál es la mejora respecto al enfoque naive y el tiempo que tarda para distintas redes.

3.7. ASV aproximado

Este algoritmo es igual al anterior. La única diferencia que tiene es respecto a cómo se calculan las clases de equivalencia, $eqClass(G, x_i)$, puesto que ahora las vamos a aproximar. Además, vamos a tener que agregar un parámetro extra, para definir la cantidad de órdenes topológicos que queremos generar. Primero utilizamos los algoritmos vistos en 3.4 para samplear los órdenes topológicos. Esto nos va a permitir calcular él ASV para polytrees o DAGs y no simplemente *dtrees*. Luego procesamos estos órdenes al igual que en la sección 3.3, para obtener las clases de equivalencia. Una vez obtenidas las clases de equivalencia, repetimos el mismo proceso del algoritmo exacto.

4. Experimental results

4.1. Experimental Evaluation

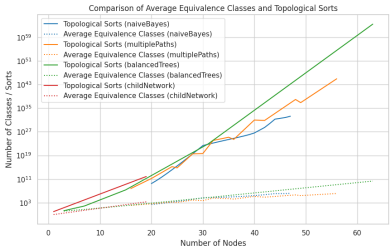
We evaluate the proposed methods by comparing them against naive baselines in terms of computational cost and scalability. Experiments were conducted on two Bayesian networks, *Cancer* and *Child*, obtained from the bnlearn repository. Both were used as data distributions and as causal DAGs. The *Child* was modified into a polytree by removing edges that introduce cycles in the underlying undirected graph.

Datasets were generated by sampling from these networks, and decision trees were trained accordingly. For ASV computation, the prediction variable was removed from the network to avoid trivial inference.

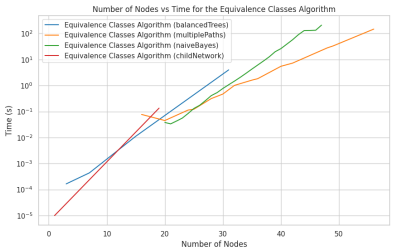
Experimental setup

- CPU: Intel i5-7500 @ 3.40GHz
- RAM: 16 GB
- OS: Ubuntu 22.04
- Python 3.12
- Libraries: pgmpy, sklearn, networkx, shap

Equivalence Classes vs. Topological Orders We compare the naive ASV computation based on all topological orders with our approach based on equivalence classes. The key quantities are the sizes of $topos(G)$ and $eqCl(G, x_i)$, as well as the cost of constructing each set.



(a) Number of equivalence classes vs. topological orders.



(b) Runtime for computing equivalence classes.

Figura 5: Comparison between equivalence classes and topological orders in terms of size and computation time.

Figure 5 shows that the number of equivalence classes grows significantly slower than the number of topological orders. For instance, in *Child*, the graph has 7.41×10^{11} topological orders but only around 2003 equivalence classes on

average. Similar gaps are observed in balanced trees, where the difference can reach several orders of magnitude.

At the same time, the cost of computing equivalence classes remains moderate for graphs of practical size, typically under a few seconds. For example, in *Child*, all equivalence classes are computed in less than one second.

This reduction directly decreases the number of evaluations of the characteristic function, making the equivalence class approach substantially more efficient.

Performance of Equivalence Class Algorithms We now analyze the cost of computing equivalence classes.

As shown in Figure 5b, the computation remains efficient for moderate graph sizes, typically under a few seconds. For example, in *Child*, all equivalence classes are computed in less than one second. However, as graph size increases, the runtime grows alongside the number of equivalence classes.

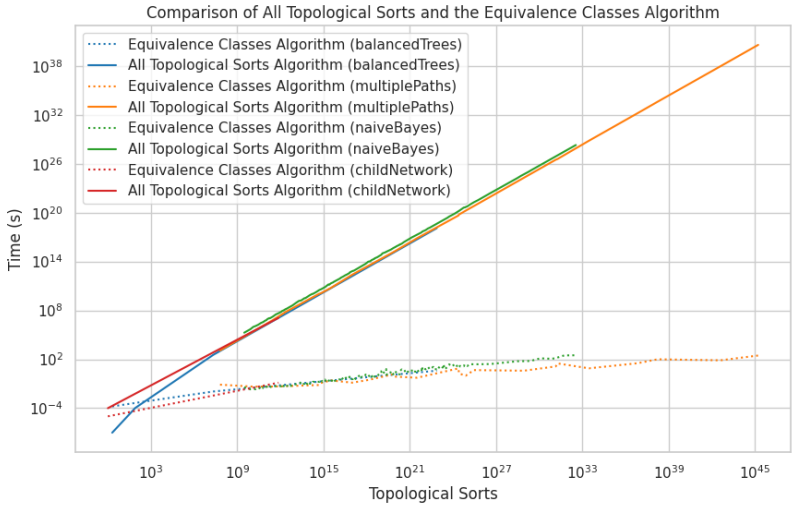


Figura 6: Runtime comparison between computing equivalence classes and enumerating all topological orders.

Figure 6 highlights the main advantage of our approach. While enumerating all topological orders quickly becomes infeasible (e.g., requiring days for large graphs), computing equivalence classes remains tractable. This is because the number of equivalence classes depends on the graph structure rather than on the total number of topological orders.

Sampling-Based Approximation of Topological Orders For graphs where equivalence classes are not directly available, we evaluate the sampling-based approach.

Number of sampled orders	**Sampling time (s)**
100	0.9270
1000	1.1170
10000	4.7170
20000	7.7170
30000	10.8387

Tabla 1: Sampling times for topological orders in *Child*.

Table 1 shows that sampling remains efficient even for large numbers of samples. Moreover, the runtime is not linear in the number of samples, as multiple candidates are generated simultaneously during recursive calls.

Finally, we evaluate the approximation error of ASV using sampled orders.

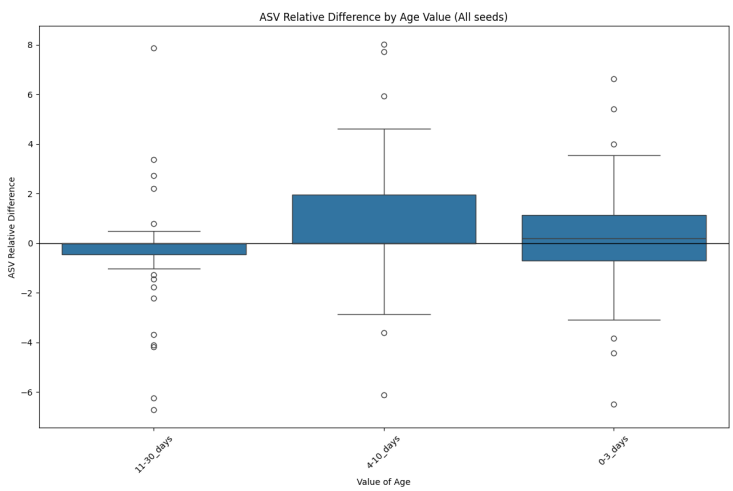


Figura 7: Relative error between approximate and exact ASV values.

Figure 7 shows that using 1000 samples yields an average error of around 8 %. Larger ASV values exhibit smaller relative error, while larger deviations correspond to very small true values.

These results indicate that sampling provides a practical trade-off between accuracy and computational cost.

Remarks on Mean Prediction Computation In our experiments, the cost of evaluating the characteristic function is not the dominant factor. For decision trees under polytree distributions, mean prediction can be computed efficiently, and thus does not limit the performance of the overall approach.

5. Conclusions

Disclosure of Interests. for example: The authors have no competing interests to declare that are relevant to the content of this article.

Referencias

Arenas, M., Barceló, P., Bertossi, L., & Monet, M. (2021). The tractability of Shap-score-based explanations for classification over deterministic and decomposable Boolean circuits. *Proceedings of the AAAI Conference on Artificial intelligence*, *35*, 6670-6678.

Arenas, M., Barceló, P., Bertossi, L., & Monet, M. (2023). On the complexity of Shap-score-based explanations: Tractability via knowledge compilation and non-approximability results. *Journal of Machine Learning Research*, *24*(63), 1-58.

Epoch AI. (2024, junio). Data on Large-Scale Ai Models [Accessed: 2025-04-14]. https://epoch.ai/data/large-scale-ai-models

et. al, E. G. (2024). FrontierMath: A Benchmark for Evaluating Advanced Mathematical Reasoning in Ai. https://arxiv.org/abs/2411.04872

Frye, C., Rowat, C., & Feige, I. (2019). Asymmetric Shapley values: incorporating causal knowledge into model-agnostic explainability. *arXiv preprint arXiv:1910.06358*.

Fryer, D., Strümke, I., & Nguyen, H. (2021). Shapley values for feature selection: The good, the bad, and the axioms. *Ieee Access*, *9*, 144352-144360.

Huang, X., & Marques-Silva, J. (2023). The inadequacy of Shapley values for explainability. *arXiv preprint arXiv:2302.08160*.

Huber, M. (2006). Fast perfect sampling from linear extensions. *Discrete Mathematics*, *306*(4), 420-428. https://doi.org/https://doi.org/10.1016/j.disc.2006.01.003

Knuth, D. E., & Szwarcfiter, J. L. (1974). A structured program to generate all topological sorting arrangements. *Information Processing Letters*, *2*(6), 153-157. https://doi.org/https://doi.org/10.1016/0020-0190(74)90001-5

Lipton, Z. C. (2017). The Mythos of Model Interpretability. https://arxiv.org/abs/1606.03490

Lundberg, S., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. https://arxiv.org/abs/1705.07874

Marques-Silva, J. (2023). Logic-based explainability in machine learning. En *Reasoning Web. Causality, Explanations and Declarative Knowledge: 18th International Summer School 2022, Berlin, Germany, September 27–30, 2022, Tutorial Lectures* (pp. 24-104). Springer.

Marzouk, R., Bassan, S., Katz, G., & de la Higuera, C. (2025). On the Computational Tractability of the (Many) Shapley Values. https://arxiv.org/abs/2502.12295

Miller, T. (2019). Explanation in Artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, *267*, 1-38. https://doi.org/https://doi.org/10.1016/j.artint.2018.07.007

Pearl, J. (1986). Fusion, Propagation, and Structuring in Belief Networks. *Artificial Intelligence*, *29*(3), 241-288.

Shapley, L. S., et al. (1953). A value for n-person games. *Classics in game theory*.

Van den Broeck, G., Lykov, A., Schleich, M., & Suciu, D. (2022). On the tractability of Shap explanations. *Journal of Artificial Intelligence Research*, *74*, 851-886.